CSE373, MAT373: Analysis of Algorithms, Spring 2024 **Date:** May 7, 2024

Final Exam

(5:30 PM – 8:00 PM : 150 Minutes)

- This exam will account for 40% of your overall grade.
- There are seven (7) questions, worth 250 points in total. Please answer all of them.
- There are 26 pages, including six (6) blank pages. Please use the blank pages if you need additional space for your answers.
- This is a *closed book*, *closed notes* exam. *No cheat sheets* are allowed.
- You are *not allowed to use calculators*.

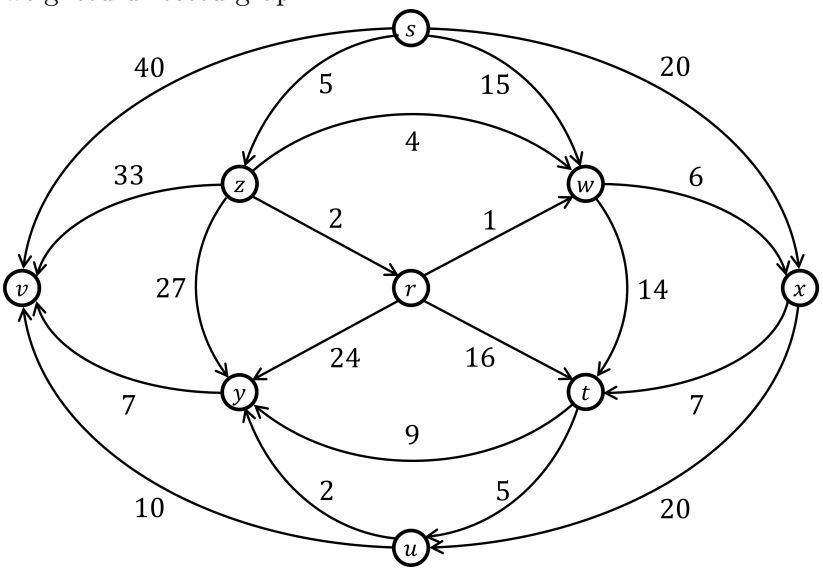
Good Luck!

Question	Parts	Points
1. Dijkstra's SSSP	(a)–(k)	$4 + 4 \times 9 + 10 = 50$
2. Kruskal's MST	(a)–(k)	$4 + 4 + 3.5 \times 8 + 4 = 40$
3. Longest Common Subsequence	(a)–(b)	$20 + 10 = 30$
4. Floyd-Warshall's APSP	(a)–(c)	$10 + 15 + 15 = 40$
5. Network Flow	(a)–(b)	$35 + 5 = 40$
6. Vertex Cover	(a)–(b)	$15 + 5 = 20$
7. Complexities	(a)–(d)	$7.5 \times 4 = 30$
Total		250

NAME: _____

SBU ID: _____

Question 1. [**50 Points**] **Dijkstra's SSSP.** Recall Dijkstra's Single-Source Shortest Paths (SSSP) algorithm we saw in the class. This task asks you for a step-by-step demonstration of how the algorithm finds shortest paths from a given source vertex to all other vertices of the following weighted directed graph.



tentative distance	predecessor
$r.d =$	$r.\pi =$
$s.d =$	$s.\pi =$
$t.d =$	$t.\pi =$
$u.d =$	$u.\pi =$
$v.d =$	$v.\pi =$
$w.d =$	$w.\pi =$
$x.d =$	$x.\pi =$
$y.d =$	$y.\pi =$
$z.d =$	$z.\pi =$

We will treat vertex s as the given source vertex. Recall that the algorithm first initializes the d and π fields of each node of the graph, and then in each step finalizes the d value of a suitable vertex and updates the d and π values of its neighbors.

(a) [**4 Points**] Initialization: What are the initial values of the d and π fields of each vertex?

(b) [**4 Points**] Step 1: Write down the 1st vertex to be finalized and the vertices whose d and π fields change (along with their new d and π values) as a result of relaxation.

(*c*) [**4 Points**] STEP 2: Repeat part (*b*) for the 2nd vertex to be finalized.

(*d*) [**4 Points**] STEP 3: Repeat part (*b*) for the 3rd vertex to be finalized.

(*e*) [**4 Points**] STEP 4: Repeat part (*b*) for the 4th vertex to be finalized.

(*f*) [**4 Points**] STEP 5: Repeat part (*b*) for the 5th vertex to be finalized.

(*g*) [**4 Points**] STEP 6: Repeat part (*b*) for the 6th vertex to be finalized.

(*h*) [**4 Points**] STEP 7: Repeat part (*b*) for the 7th vertex to be finalized.

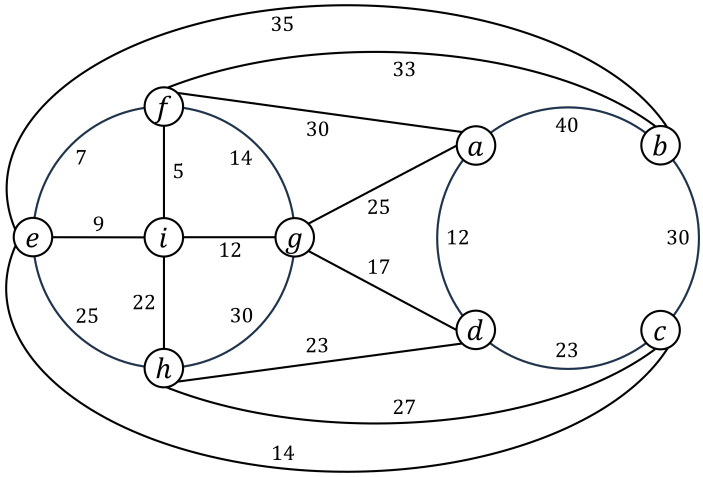
(*i*) [**4 Points**] STEP 8: Repeat part (*b*) for the 8th vertex to be finalized.

(*j*) [**4 Points**] STEP 9: Repeat part (*b*) for the 9th vertex to be finalized.

(*k*) [**10 Points**] By examining the final d and π fields of the vertices of the graph, write down the shortest distance and the corresponding shortest path from the source vertex (i.e., vertex s) to each of the other vertices.

page intentionally left blank (use for your answers, if needed)

Question 2. [**40 Points**] **Kruskal's MST.** Recall Kruskal's Minimum Spanning Tree (MST) algorithm we saw in the class. This task asks you for a step-by-step demonstration of how the algorithm finds an MST of the following weighted undirected graph.



Recall that the algorithm examines the edges one by one in a predetermined sequence. In each step the algorithm examines one edge, and decides to either include it in the MST or discard it.

(a) [**4 Points**] Give an order in which the algorithm may examine the edges of the graph.

(b) [**4 Points**] In Kruskal's algorithm when do you decide not to include an edge to the MST?

(*c*) [**3.5 Points**] Write down the 1st edge to be included in the MST.

(*d*) [**3.5 Points**] Write down the 2nd edge to be included in the MST.

(*e*) [**3.5 Points**] Write down the 3rd edge to be included in the MST.

(*f*) [**3.5 Points**] Write down the 4th edge to be included in the MST.

(*g*) [**3.5 Points**] Write down the 5th edge to be included in the MST.

(*h*) [**3.5 Points**] Write down the 6th edge to be included in the MST.

(i) [**3.5 Points**] Write down the 7th edge to be included in the MST.

(j) [**3.5 Points**] Write down the 8th edge to be included in the MST.

(k) [**4 Points**] What is the weight of the MST you have just computed?

page intentionally left blank (use for your answers, if needed)

Question 3. [**30 Points**] **Longest Common Subsequence.** This task asks you to find the longest common subsequence (LCS) between two given sequences using the dynamic programming algorithm we saw in the class.

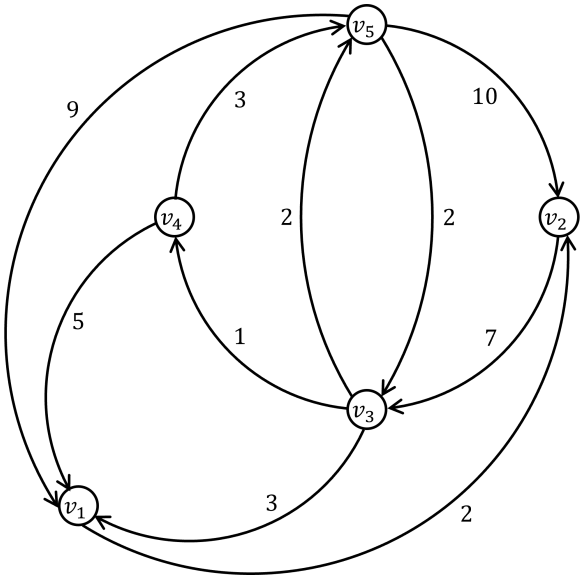
You will have to fill out the following dynamic programming table as we did in the class. For each cell in the table you will have to compute the value of the cell and choose an appropriate arrow (up: $\uparrow$, left: $\leftarrow$, up-left: $\nwarrow$) to point to the neighboring cell from which the value of the current cell has been computed.

	j	0	1	2	3	4	5	6	7	8
i		y_j	G	A	C	T	C	T	A	G
0	x_i									
1	G									
2	C									
3	T									
4	A									
5	C									
6	T									
7	G									

(*a*) [**20 Points**] Fill out the table given above, and report the length of an LCS.

(*b*) [**10 Points**] What is the LCS computed by the algorithm? Explain how you find it from the table.

Question 4. [**40 Points**] **Floyd-Warshall's APSP.** This task asks you to run Floyd-Warshall's all-pairs shortest paths (APSP) algorithm on the following weighted directed graph.



The algorithm starts with an initial distance matrix $D^{(0)}$ and predecessor matrix $\Pi^{(0)}$, and then for each $k \in [1, n]$ it computes, $D^{(k)}$ and $\Pi^{(k)}$ from $D^{(k-1)}$ and $\Pi^{(k-1)}$. It returns $D^{(n)}$ and $\Pi^{(n)}$ as its final answer.

In this questions we are interested in $D^{(k)}$ and $\Pi^{(k)}$ matrices for $k \in [0, 2]$ only.

(a) [**10 Points**] Fill out the $D^{(0)}$ and $\Pi^{(0)}$ matrices below based on the input graph given above.

$D^{(0)} =$

	v_1	v_2	v_3	v_4	v_5
v_1					
v_2					
v_3					
v_4					
v_5					

$\Pi^{(0)} =$

	v_1	v_2	v_3	v_4	v_5
v_1					
v_2					
v_3					
v_4					
v_5					

(*b*) [**15 Points**] Fill out the $D^{(1)}$ and $\Pi^{(1)}$ matrices given below based on what you computed in part (*a*).

$D^{(1)} =$

	v_1	v_2	v_3	v_4	v_5
v_1					
v_2					
v_3					
v_4					
v_5					

$\Pi^{(1)} =$

	v_1	v_2	v_3	v_4	v_5
v_1					
v_2					
v_3					
v_4					
v_5					

(*c*) [**15 Points**] Fill out the $D^{(2)}$ and $\Pi^{(2)}$ matrices given below based on what you computed in part (*b*).

$D^{(2)} =$

	v_1	v_2	v_3	v_4	v_5
v_1					
v_2					
v_3					
v_4					
v_5					

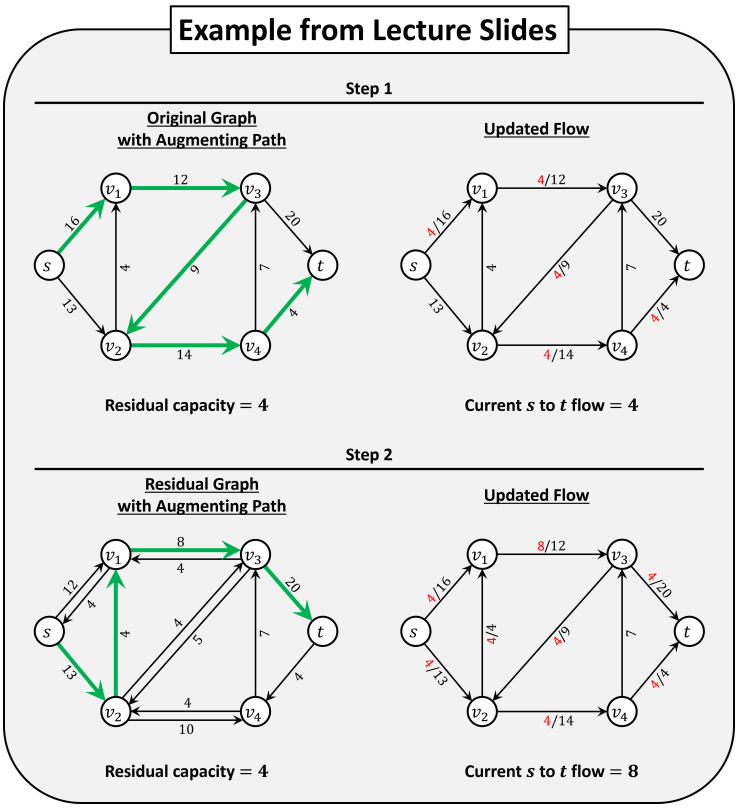
$\Pi^{(2)} =$

	v_1	v_2	v_3	v_4	v_5
v_1					
v_2					
v_3					
v_4					
v_5					

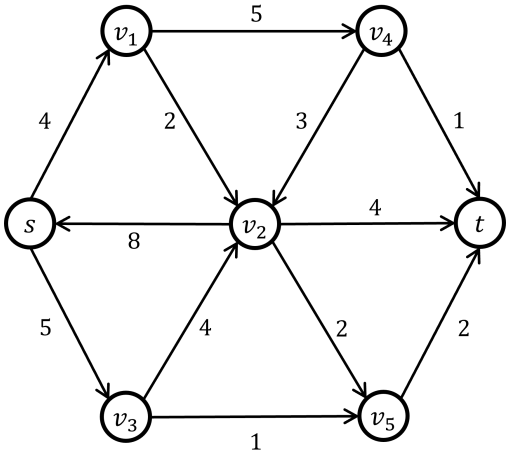
page intentionally left blank (use for your answers, if needed)

Question 5. [40 Points] Network Flow. This task asks you to run Ford-Fulkerson's method for computing a maximum flow from a source vertex s to a sink vertex t in a directed graph (network) with capacities.

The method executes multiple steps. In every step it finds an augmenting path either in the input graph (in step 1) or in the residual graph (in all other steps) and updates flows in the given graph based on the flow through that path. It stops when no more augmenting paths can be found. The following example shows the first two steps of the method on the graph from the lecture slides, where the thick edges make the augmenting path.



Your input graph for this question is given below.



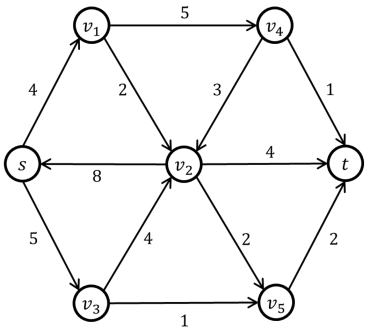
(*a*) [**35 Points**] Apply the Ford-Fulkerson's method on the input graph for as many steps as necessary (adding additional steps, if required) to find a maximum s to t flow.

In each step, show your augmenting path (with thick edges) on the left and the updated flows on the right. Also write down the residual capacity and the current s to t flow in the network.

Starting from step 2, construct the residual graph on the left by adding directions to edges and adding additional edges as necessary.

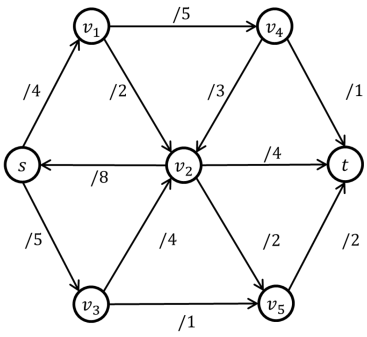
Step 1

Original Graph with Augmenting Path



Residual capacity =

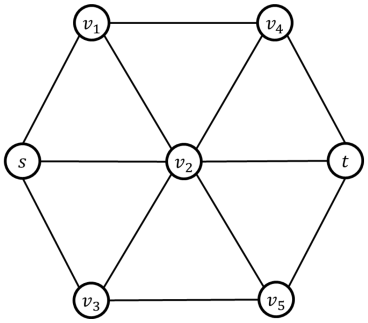
Updated Flow



Current s to t flow =

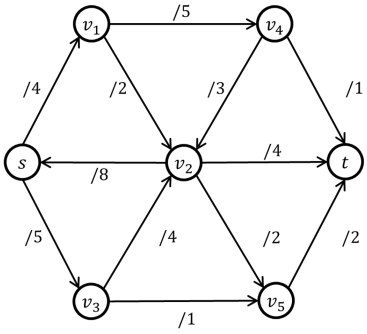
Step 2

Residual Graph with Augmenting Path



Residual capacity $=$

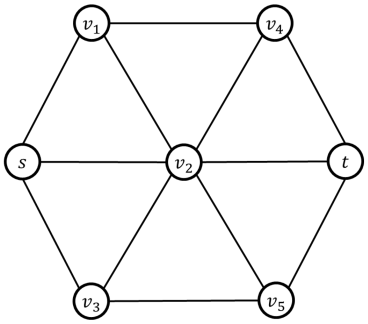
Updated Flow



Current s to t flow $=$

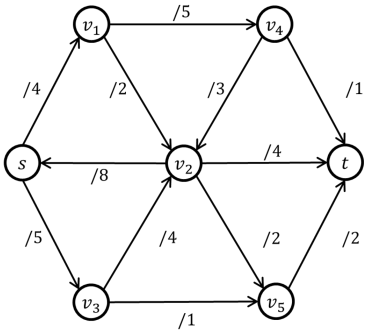
Step 3

Residual Graph with Augmenting Path



Residual capacity $=$

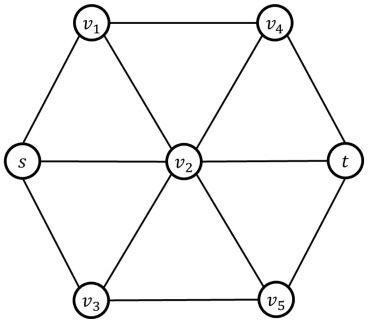
Updated Flow



Current s to t flow $=$

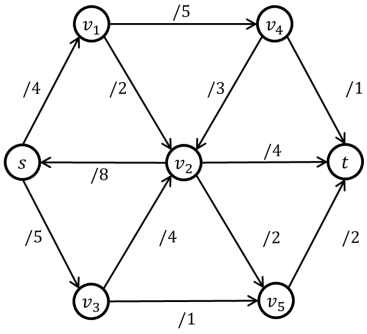
Step 4

Residual Graph with Augmenting Path



Residual capacity =

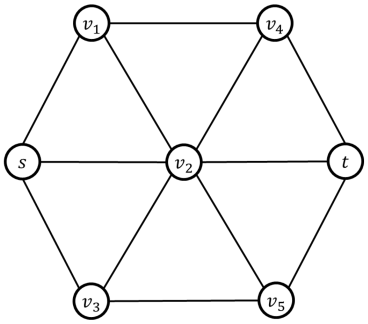
Updated Flow



Current s to t flow =

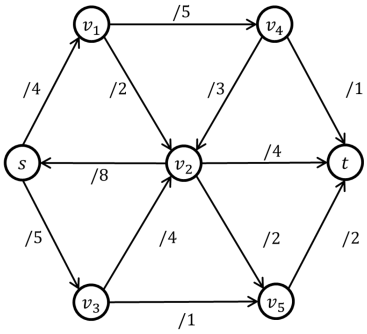
Step 5

Residual Graph with Augmenting Path



Residual capacity =

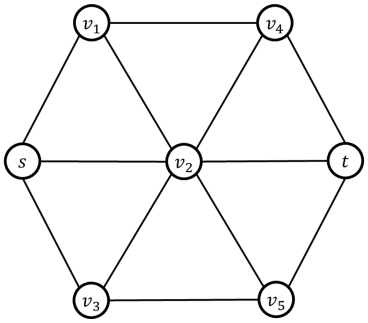
Updated Flow



Current s to t flow =

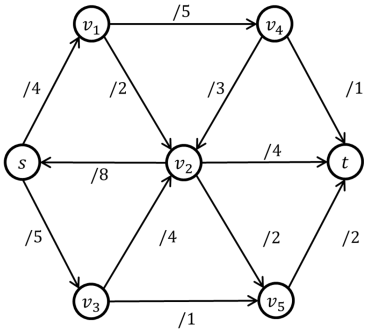
Step 6

Residual Graph with Augmenting Path



Residual capacity =

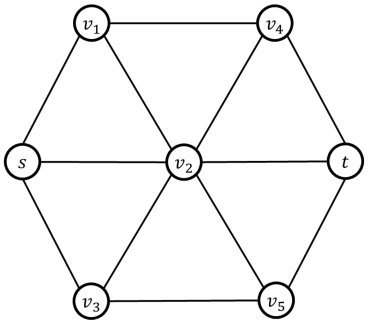
Updated Flow



Current s to t flow =

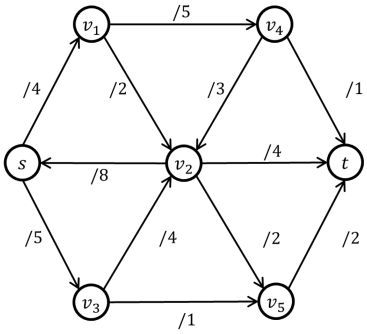
Step 7

Residual Graph with Augmenting Path



Residual capacity =

Updated Flow



Current s to t flow =

(*b*) [**5 Points**] What is the maximum s to t flow you computed in part (*a*)?

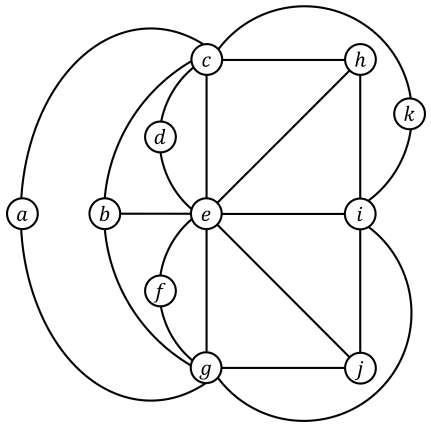
page intentionally left blank (use for your answers, if needed)

page intentionally left blank (use for your answers, if needed)

Question 6. [**20 Points**] **Vertex Cover.** This task asks you to find an approximate vertex cover of an undirected graph using the 2-approximate vertex cover algorithm we saw in the class.

The algorithm executes multiple steps. It starts with an empty vertex cover C. In every step it selects an edge of the graph, adds some vertices to C, and removes some edges from the graph. It stops when all edges of the graph are removed, and returns C as the approximate vertex cover.

Your input graph for this question is given below.



(a) [**15 Points**] Run the approximate vertex cover algorithm on the graph given above until all edges are removed from it. Add additional steps, if necessary.

Step 1:

Edge chosen:

Vertices added to C:

Edges removed:

Step 2:

Edge chosen:

Vertices added to C:

Edges removed:

Step 3:

Edge chosen:

Vertices added to C:

Edges removed:

(b) [**5 Points**] What is the approximate vertex cover you computed in part (a)?

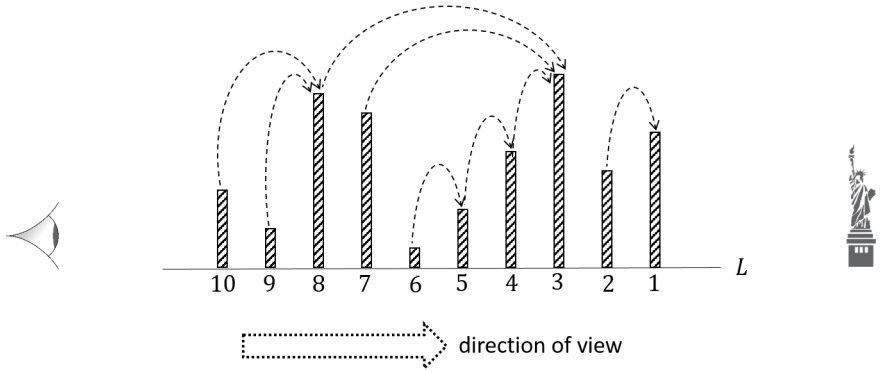
Question 7. [**30 Points**] **Complexities.** For each of the problems given below, write down which problem we saw in the class it corresponds to. If there is a polynomial-time (or pseudo-polynomial time) algorithm for solving the problem, state the running time of that algorithm. Otherwise write "`Hard`" instead of stating the running time.

No justifications necessary, but you must state what each variable in your stated complexity means (e.g., meanings of m and n in $\mathcal{O}((m+n)\log\log m)$).

(*a*) [**7.5 Points**] You are given a set B of n blocks. You are also given a set S of all *conflicting pairs* of blocks $\langle b_i, b_j \rangle$ with $b_i, b_j \in B$, such that b_i and b_j cannot be put side by side. Now, you want to find out if it is possible to arrange the n blocks in a circle so that no two adjacent blocks form a conflicting pair.

(*b*) [**7.5 Points**] A book store has exactly n books with $p_1, p_2, \ldots, p_n$ being their prices, where each p_i is a positive integer. You have exactly m amount of money, where m is also an integer. You want to know if there exists a subset of those books whose prices add up to exactly m.

(*c*) [**7.5 Points**] You are given a set A of n activities. You are also given a set S of all pairs of activities $\langle a_i, a_j \rangle$ with $a_i, a_j \in A$, such that a_i and a_j cannot be performed simultaneously. Now, you want to find the largest subset of activities $A' \subseteq A$ such that no two activities in A' can be performed simultaneously.



(*d*) [**7.5 Points**] Manhattan is full of tall buildings, and even if you are on the top of a very tall building, often another building that is even taller will obstruct your view. Now suppose you are given n buildings on an imaginary straight line[1] with the statue of liberty at one end. For every building, you want to know if someone stands on top of that building and looks in the direction of the statue how many buildings taller than the building the person is standing on will be visible to that person.

[1] line of sight

page intentionally left blank (use for your answers, if needed)